

Experiment 2

Create a Blockchain using Python

Aim

To create a Blockchain in Python with block creation, Proof-of-Work mining, chain validation, and pending-transaction handling, and to demonstrate its working over a Flask HTTP API.

Theory

A blockchain is a distributed, append-only ledger made of blocks linked by hash: each block stores an index, a timestamp, a Proof-of-Work value, the hash of the previous block, and the transactions it contains. Linking blocks by the previous block's hash is what makes the chain tamper-evident — changing any historical block changes its hash, which breaks every `previous_hash` reference after it.

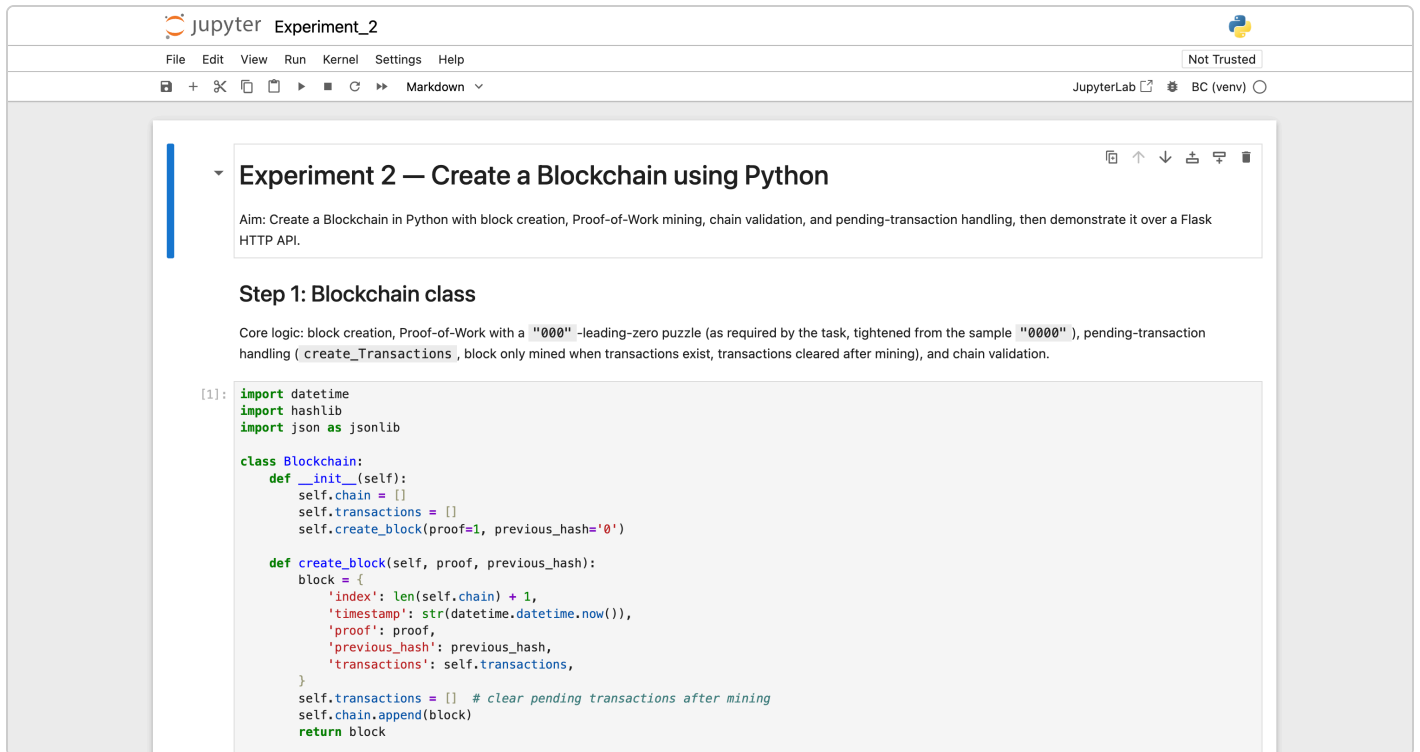
Building on Experiment 1's Proof-of-Work primitive, this experiment wraps it in a full `Blockchain` class with three additional behaviors required by the task: (1) a `create_Transactions` method that queues transactions instead of writing them straight into a block, (2) mining that is gated on the pending-transaction pool being non-empty, so no Proof-of-Work is wasted mining an empty block, and (3) the pending pool being cleared as part of `create_block`, so a mined transaction cannot be mined again. The Proof-of-Work target was also tightened from the sample `"0000"` (4 leading zeros) to the task-specified `"000"` (3 leading zeros), which lowers the average number of nonce trials needed per block.

The same `Blockchain` class is then exposed over HTTP with Flask, using `/mine_block`, `/add_transaction`, `/get_chain`, and `/is_valid` routes — the same interface a Postman session would exercise, driven here instead with Python's `requests` library so the whole demonstration is reproducible from one notebook run.

Tools used: Python 3.12, `hashlib`, `datetime`, Flask, `requests`, run in a local Jupyter Notebook (`Experiment_2.ipynb`, in this folder) with the Flask server running in a background thread in place of

a separate terminal + Postman session.

Output



The screenshot shows a JupyterLab environment with a notebook titled "Experiment 2 — Create a Blockchain using Python". The notebook's aim is to create a blockchain in Python, including block creation, Proof-of-Work mining, chain validation, and pending-transaction handling, demonstrated over a Flask HTTP API. The first step is defining a "Blockchain class". The core logic involves block creation with a 3-leading-zero puzzle, pending-transaction handling (using create_Transactions), block mining (only when transactions exist), and chain validation. The code in the first cell imports datetime, hashlib, and json as jsonlib, and defines the Blockchain class with methods for initialization, block creation, and transaction handling.

```
[1]: import datetime
import hashlib
import json as jsonlib

class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(proof=1, previous_hash='0')

    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': self.transactions,
        }
        self.transactions = [] # clear pending transactions after mining
        self.chain.append(block)
        return block
```

Step 1: the `Blockchain` class with `create_block`, `create_Transactions`, `proof_of_work` (3-leading-zero target), `hash`, and `is_chain_valid`.

```
jupyter Experiment_2
File Edit View Run Kernel Settings Help
JupyterLab BC (venv)

def create_transactions(self, sender, receiver, amount):
    self.transactions.append({'sender': sender, 'receiver': receiver, 'amount': amount})
    return self.get_previous_block()['index'] + 1

def get_previous_block(self):
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    new_proof = 1
    check_proof = False
    while not check_proof:
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()
        if hash_operation[:3] == '000': # golden-nonce puzzle: 3 leading zeros
            check_proof = True
        else:
            new_proof += 1
    return new_proof

def hash(self, block):
    encoded_block = jsonlib.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(
            str(proof**2 - previous_proof**2).encode()
        ).hexdigest()
        if hash_operation[:3] != '000':
            return False
        block_index += 1
```

Continuation of Step 1: the full `proof_of_work`, `hash`, and `is_chain_valid` methods, ending with the genesis block being created.

```
jupyter Experiment_2
File Edit View Run Kernel Settings Help
JupyterLab BC (venv)

    previous_block = block
    block_index += 1
    return True

blockchain = Blockchain()
print("Genesis block created:", blockchain.chain[0])

Genesis block created: {'index': 1, 'timestamp': '2026-09-09 11:49:36.507440', 'proof': 1, 'previous_hash': '0', 'transactions': []}

Step 2: Mine only when transactions are pending
Mining is gated on self.transactions being non-empty, and the pending list is cleared as part of create_block once mined.

[2]: def mine_block(bc):
    if not bc.transactions:
        return {'message': "No pending transactions - nothing to mine."}
    previous_block = bc.get_previous_block()
    previous_proof = previous_block['proof']
    proof = bc.proof_of_work(previous_proof)
    previous_hash = bc.hash(previous_block)
    block = bc.create_block(proof, previous_hash)
    return {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions'],
    }

    # Attempt to mine with no pending transactions
    print("Mine with empty pool:", mine_block(blockchain))

    # Add transactions, then mine
    blockchain.create_transactions("Alice", "Bob", 200)
    blockchain.create_transactions("Bob", "Dave", 500)
    print("Pending before mining:", blockchain.transactions)
```

Step 2: `mine_block` refuses to mine when the transaction pool is empty, then successfully mines a block once 2 transactions are queued — the pool is empty again immediately after.

```
jupyter Experiment_2
File Edit View Run Kernel Settings Help
Not Trusted
JupyterLab BC (venv)

print("Mine result:", mine_block(blockchain))
print("Pending after mining :", blockchain.transactions)

Mine with empty pool: {'message': 'No pending transactions – nothing to mine.'}
Pending before mining: [{'sender': 'Alice', 'receiver': 'Bob', 'amount': 200}, {'sender': 'Bob', 'receiver': 'Dave', 'amount': 500}]
Mine result: {'message': 'Congratulations, you just mined a block!', 'index': 2, 'timestamp': '2026-09-09 11:49:36.513938', 'proof': 533, 'previous_hash': '43e7548729265b70730c0b4de2eaa19dc5716167ce2d39f23c25341fe5ee770f', 'transactions': [{'sender': 'Alice', 'receiver': 'Bob', 'amount': 200}, {'sender': 'Bob', 'receiver': 'Dave', 'amount': 500}]}
Pending after mining : []

Step 3: Inspect the chain and validate it

[3]: for block in blockchain.chain:
      print(block)

      print("\nIs chain valid?", blockchain.is_chain_valid(blockchain.chain))

{'index': 1, 'timestamp': '2026-09-09 11:49:36.507440', 'proof': 1, 'previous_hash': '0', 'transactions': []}
{'index': 2, 'timestamp': '2026-09-09 11:49:36.513938', 'proof': 533, 'previous_hash': '43e7548729265b70730c0b4de2eaa19dc5716167ce2d39f23c25341fe5ee770f', 'transactions': [{'sender': 'Alice', 'receiver': 'Bob', 'amount': 200}, {'sender': 'Bob', 'receiver': 'Dave', 'amount': 500}]}

Is chain valid? True

Step 4: Expose the Blockchain over a Flask API

The same class is wrapped in a Flask app with /mine_block, /get_chain, and /is_valid routes, run in a background thread inside this notebook so it can be exercised with real HTTP requests (in place of a separate Postman session).

[4]: from flask import Flask, jsonify, request
      import threading

      app = Flask(__name__)
      api_blockchain = Blockchain()
```

Step 3: both blocks printed and the chain confirmed valid. Step 4 begins: the same Blockchain wrapped in a Flask app with `/mine_block`, `/add_transaction`, `/get_chain`, `/is_valid` routes.

```
jupyter Experiment_2
File Edit View Run Kernel Settings Help
Not Trusted
JupyterLab BC (venv)

@app.route('/mine_block', methods=['GET'])
def route_mine_block():
    return jsonify(mine_block(api_blockchain)), 200

@app.route('/add_transaction', methods=['POST'])
def route_add_transaction():
    data = request.get_json()
    index = api_blockchain.create_transactions(data['sender'], data['receiver'], data['amount'])
    return jsonify({'message': f'Transaction will be added to Block {index}'}, 201)

@app.route('/get_chain', methods=['GET'])
def route_get_chain():
    return jsonify({'chain': api_blockchain.chain, 'length': len(api_blockchain.chain)}, 200)

@app.route('/is_valid', methods=['GET'])
def route_is_valid():
    valid = api_blockchain.is_chain_valid(api_blockchain.chain)
    message = 'All good. The Blockchain is valid.' if valid else 'Houston, we have a problem. The Blockchain is not valid.'
    return jsonify({'message': message}), 200

server_thread = threading.Thread(
    target=lambda: app.run(host='127.0.0.1', port=5050, use_reloader=False),
    daemon=True,
)
server_thread.start()
import time; time.sleep(1)
print("Flask API running on http://127.0.0.1:5050")

* Serving Flask app '__main__'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5050
Press CTRL+C to quit
Flask API running on http://127.0.0.1:5050

Step 5: Exercise the API with requests (in place of Postman)
```

The four Flask routes and the background server thread starting successfully on `http://127.0.0.1:5050`.

```
jupyter Experiment_2
File Edit View Run Kernel Settings Help
Not Trusted
JupyterLab BC (venv)

base = "http://127.0.0.1:5050"

r = requests.get(f"{base}/mine_block")
print("GET /mine_block (no transactions yet):", r.json())

r = requests.post(f"{base}/add_transaction", json={"sender": "Alice", "receiver": "Bob", "amount": 200})
print("POST /add_transaction:", r.json())

r = requests.post(f"{base}/add_transaction", json={"sender": "Bob", "receiver": "Dave", "amount": 500})
print("POST /add_transaction:", r.json())

r = requests.get(f"{base}/mine_block")
print("GET /mine_block (with transactions):", r.json())

r = requests.get(f"{base}/get_chain")
print("GET /get_chain:", r.json())

r = requests.get(f"{base}/is_valid")
print("GET /is_valid:", r.json())

127.0.0.1 - - [09/Sep/2026 11:49:37] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [09/Sep/2026 11:49:38] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [09/Sep/2026 11:49:38] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [09/Sep/2026 11:49:38] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [09/Sep/2026 11:49:38] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [09/Sep/2026 11:49:38] "GET /is_valid HTTP/1.1" 200 -
GET /mine_block (no transactions yet): {'message': 'No pending transactions - nothing to mine.'}
POST /add_transaction: {'message': 'Transaction will be added to Block 2'}
POST /add_transaction: {'message': 'Transaction will be added to Block 2'}
GET /mine_block (with transactions): {'index': 2, 'message': 'Congratulations, you just mined a block!', 'previous_hash': '951ab7233d17ca22234dcfd86fc011dd45c537d2e65994bc952a68a4bfc301d9', 'proof': 533, 'timestamp': '2026-09-09 11:49:38.102074', 'transactions': [{'amount': 200, 'receiver': 'Bob', 'sender': 'Alice'}, {'amount': 500, 'receiver': 'Dave', 'sender': 'Bob'}]}
GET /get_chain: {'chain': [{'index': 1, 'previous_hash': '0', 'proof': 1, 'timestamp': '2026-09-09 11:49:36.632923', 'transactions': []}, {'index': 2, 'previous_hash': '951ab7233d17ca22234dcfd86fc011dd45c537d2e65994bc952a68a4bfc301d9', 'proof': 533, 'timestamp': '2026-09-09 11:49:38.102074', 'transactions': [{'amount': 200, 'receiver': 'Bob', 'sender': 'Alice'}, {'amount': 500, 'receiver': 'Dave', 'sender': 'Bob'}]}], 'length': 2}
GET /is_valid: {'message': 'All good. The Blockchain is valid.'}
```

Step 5: the API driven with `requests` — mining with no transactions returns “nothing to mine”, two transactions are POSTed, then mining succeeds and `/get_chain` and `/is_valid` confirm a 2-block valid chain, mirroring exactly what Postman would show.

Conclusion

This experiment built a complete, working Blockchain in Python and exposed it as a real HTTP service. Tightening the Proof-of-Work target from 4 to 3 leading zeros ("000") visibly reduced the mining difficulty — the golden-nonce search in this run resolved in 533 attempts versus the tens of thousands seen for a 4-zero target in Experiment 1. Gating `mine_block` on a non-empty transaction pool and clearing that pool inside `create_block` together prevent both wasted mining (on empty blocks) and double-mining (of the same transactions). Finally, driving the Flask `/mine_block`, `/add_transaction`, `/get_chain`, and `/is_valid` endpoints with the `requests` library reproduced exactly what a Postman session would show — confirming the chain stayed valid (`is_chain_valid` returned `True`) after mining — while keeping the whole demonstration self-contained and reproducible in one notebook run.